

GitHub Actions e GitLab CI

Orquestração de CI/CD Moderna para Desenvolvimento de Software

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Sumário

- Orquestração CI/CD Moderna
- Estrutura de Arquivos e Ecossistemas
- Hospedagem e Infraestrutura
- Prática: Sintaxe GitHub Actions e GitLab CI
- Comparativo e Critérios de Escolha

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Orquestração de CI/CD Moderna

GitHub Actions e GitLab CI representam o estado da arte na automação e orquestração de pipelines de desenvolvimento. Ambas as plataformas utilizam o paradigma '*Pipeline as Code*', onde a lógica de automação é definida em arquivos YAML no repositório.

Pontos fundamentais:

- Automação de processos de Build, Test e Deploy.
- Integração nativa com o controle de versão.
- Orquestração baseada em eventos (push, merge request, tags).
- Escalabilidade através de executores gerenciados ou próprios.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



GitHub Actions

A estrutura de arquivos é **modular** e organizada no diretório ``.github/workflows/``. Permite a criação de múltiplos arquivos YAML, facilitando a separação de responsabilidades (ex: um workflow para testes, outro para deploy), o que foi uma evolução natural da nossa discussão sobre Pipeline as Code.

GitLab CI

A centralização é o foco, utilizando um único arquivo obrigatório chamado ``.gitlab-ci.yml``, localizado na raiz do projeto. Embora permita a inclusão de outros arquivos via palavra-chave ``include``, a entrada principal para a definição do pipeline permanece única.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Ecossistemas: Marketplace vs. Scripts

GitHub Marketplace

Foca no **reuso de componentes** de alto nível e 'Actions' prontas criadas pela comunidade para agilizar o desenvolvimento.

GitLab CI Nativo

Prioriza o **controle granular** através de scripts shell nativos, imagens Docker customizadas e templates integrados.

Padronização e ConformidadeA

Enquanto o GitHub favorece a agilidade via Marketplace, o GitLab oferece ferramentas robustas para governança corporativa e templates globais.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Hospedagem e Infraestrutura

GitHub Cloud

Fortemente centrado no modelo *SaaS* (Software as a Service), oferecendo **runners gerenciados** prontos para uso. Embora permita a configuração de 'self-hosted runners' para necessidades específicas, a ênfase é na conveniência da nuvem.

GitLab Runners

O GitLab Runner é um agente leve e **versátil**, projetado para ser *híbrido*. Ele funciona perfeitamente em ambientes *on-premises*, nuvens privadas ou em clusters **Kubernetes**, oferecendo grande flexibilidade de implantação.

Flexibilidade Corporativa

O GitLab se destaca em ambientes corporativos que exigem **isolamento total** de rede ou hardware específico. Isso inclui cenários atrás de firewalls e em redes sem acesso direto à internet, onde a segurança e o controle são primordiais.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Demonstração Guiada

Nesta demonstração prática, exploraremos a sintaxe YAML para configurar uma esteira de integração contínua (CI) básica no GitHub, focando na estrutura essencial de um workflow de GitHub Actions. Usaremos como base o seguinte trecho de código:

```
name: CI Pipeline
on: push: branches: [main]
```

E detalharemos os componentes de um job, como runs-on: ubuntu-latest, steps, steps: [ubuntu-latest], e steps: [ubuntu-latest, ubuntu-latest].

Para este workflow, além do name (identificação) e do on (gatilho de disparo), precisamos de um bloco jobs para definir as tarefas. Dentro de um job, o runs-on: ubuntu-latest especifica o ambiente virtual (runner) que executará os comandos. Já o steps é uma lista ordenada de tarefas — diferente do que parece, o ubuntu-latest não vai dentro dele. Na prática, utilizamos uses para reaproveitar actions prontas (como actions/checkout@v4) e run para comandos shell diretos (ex: run: npm install). Durante a demonstração, evoluiremos esse esqueleto inicial adicionando um job completo com checkout do código, instalação de dependências e execução de testes, mostrando como cada peça se comporta na esteira de CI.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Demonstração Guiada

- 1 Começaremos definindo o nome do nosso workflow usando ``name: CI Pipeline``. Este campo é crucial para identificar facilmente seu pipeline na interface do GitHub, fornecendo uma descrição clara de sua finalidade.
- 2 Em seguida, especificamos o *gatilho* (trigger) do workflow com ``on: push: branches: [ main ]``. Isso instrui o GitHub Actions a executar este pipeline sempre que ocorrer um ``push`` (envio de código) para o branch ``main``, garantindo que o CI seja acionado automaticamente em cada atualização importante.
- 3 Definimos o ambiente de execução para os jobs do nosso workflow usando ``runs-on: ubuntu-latest``. Esta configuração é essencial porque determina o sistema operacional e a versão da máquina virtual onde suas ações serão executadas, assegurando a compatibilidade com suas dependências de projeto.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Demonstração Guiada

- 4 Introduzimos o bloco ``steps``, que é uma sequência ordenada de tarefas ou comandos a serem executados. Cada *step* representa uma etapa lógica no processo do CI, desde a instalação de dependências até a execução de testes, e sua ordem é fundamental para o fluxo do pipeline.
- 5 Utilizamos a palavra-chave ``uses`` para invocar ações pré-definidas do Marketplace do GitHub Actions, como ``actions/checkout@v4``. Esta é uma prática recomendada para reutilizar funcionalidades comuns, como clonar o repositório (``checkout``), sem precisar reescrever o código para cada workflow.
- 6 Empregamos a palavra-chave ``run`` para executar comandos de shell diretamente no ambiente de execução. Por exemplo, podemos ter ``run: npm install`` para instalar as dependências do projeto ou ``run: npm test`` para executar os testes unitários. Isso nos permite personalizar as etapas do CI com comandos específicos da nossa aplicação.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Demonstração Guiada

Continuando nossa prática de CI/CD, agora vamos demonstrar a sintaxe YAML no GitLab CI, focando na definição de estágios (stages) e jobs. Veremos como organizar o pipeline para o fluxo de build e teste de uma aplicação. No GitLab CI, começamos definindo os estágios em um bloco stages, como em: stages: - build -test. Após definir os estágios, cada job é vinculado a um deles pela chave stage. Assim, um job build-job usa stage: build, e o test-job usa stage: test. Por padrão, jobs do mesmo estágio rodam em paralelo, enquanto estágios distintos executam em sequência — o que significa que o test só dispara após o build ser concluído com êxito. Durante a demonstração, construiremos um pipeline prático com um job que instala dependências e compila o código, e outro que roda os testes automatizados. Vamos acompanhar a execução passo a passo pela interface do GitLab, validando a ordem de execução e a lógica de falha rápida (fail-fast) entre os estágios.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Demonstração Guiada

1

Primeiro, defina os estágios globais do pipeline. No GitLab CI, você usa a palavra-chave ``stages`` para listar os estágios que seu pipeline terá. A ordem em que os estágios são listados é a ordem de execução. Por exemplo, ``build`` virá antes de ``test``. Isso garante que o build seja concluído antes que os testes possam começar, refletindo um fluxo de trabalho sequencial e lógico.

2

Em seguida, defina um job para o estágio de build. Crie um job nomeado ``build_job`` e associe-o ao estágio ``build`` usando a palavra-chave ``stage: build``. Dentro deste job, utilize a seção ``script`` para especificar os comandos a serem executados. Por exemplo, ``npm install`` é o comando que instala as dependências do projeto. Este passo garante que seu ambiente de build esteja pronto para compilar o código.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Demonstração Guiada

3 Logo após, defina um job para o estágio de teste. Crie um job ``test_job`` e associe-o ao estágio ``test`` com ``stage: test``. O comando ``npm test`` na seção ``script`` será responsável por executar os testes unitários ou de integração do seu projeto. A associação clara a um estágio garante que este job só será executado após a conclusão bem-sucedida do estágio anterior (build), mantendo a integridade do pipeline.

4 Observe que no GitLab CI, a ordem de execução do pipeline é **explicitamente definida pela lista de `stages`**. Isso difere de outras plataformas onde a ordem pode ser inferida ou definida por dependências de jobs. Além disso, jobs pertencentes ao *mesmo estágio* são executados em **paralelo por padrão**, otimizando o tempo de execução do pipeline ao permitir que tarefas independentes sejam processadas simultaneamente.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Comparativo de Funcionalidades

Esta tabela detalha as principais características comparativas entre GitHub Actions e GitLab CI.

Característica	GitHub Actions	GitLab CI
Configuração	Múltiplos arquivos em <code>`.github/workflows/`</code>	Único arquivo <code>`.gitlab-ci.yml`</code>
Extensibilidade	GitHub Marketplace (Actions)	Templates e Imagens Docker
Execução	Runners (foco em Cloud)	Runners (foco em Self-hosted)
Complexidade	Curva de aprendizado inicial baixa	Controle granular avançado
Orquestração	Baseada em passos e jobs	Baseada em estágios (stages)

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Discuta com o colega ao lado

Quais fatores sopesariam sua decisão entre GitHub Actions e GitLab CI em um projeto real?



Considere requisitos de segurança, familiaridade da equipe e custos de manutenção vs. nuvem.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Verificação de Aprendizado. Qual das seguintes afirmações descreve corretamente uma diferença fundamental entre as duas

- A O GitLab CI organiza a execução através de 'stages' definidos no arquivo .gitlab-ci.yml.
- B O GitLab CI utiliza o Marketplace como principal forma de reutilizar lógica de pipeline.
- C O GitHub Actions não permite a execução de comandos shell (run), apenas o uso de Actions prontas.
- D O GitHub Actions exige que todos os workflows estejam em um único arquivo chamado pipeline.yml.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Verificação de Aprendizizado. Qual das seguintes afirmações descreve corretamente uma diferença fundamental entre as duas ferramentas?

- ✓ O GitLab CI organiza a execução através de 'stages' definidos no arquivo .gitlab-ci.yml.
- ✗ O GitLab CI utiliza o Marketplace como principal forma de reutilizar lógica de pipeline.
- ✗ O GitHub Actions não permite a execução de comandos shell (run), apenas o uso de Actions prontas.
- ✗ O GitHub Actions exige que todos os workflows estejam em um único arquivo chamado pipeline.yml.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Conclusão

- GitHub Actions favorece agilidade com Marketplace e múltiplos arquivos YAML, ideal para integração nativa com o GitHub.
- GitLab CI prioriza controle granular via scripts, estágios e runners híbridos, adequado para governança corporativa.
- A escolha depende de auto-hospedagem, familiaridade da equipe e requisitos de segurança/isolamento da infraestrutura.
- Ambas as plataformas utilizam 'Pipeline as Code' em YAML, definindo automação de build, test e deploy.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

